

۱: مبانی جاوا (Java Fundamentals) - کامل و

عمیق

این بخش، سنگ بنای همه چیزه. اگر این مفاهیم رو عمیق نفهمی، توی مراحل بعدی (مخصوصاً OOP و کالکشن‌ها) به مشکل میخوری. توی مصاحبه‌های اینترپرایز هم از همین مبانی کلی سوال میپرسن.

1.1 انواع داده (Data Types)

جاوا یه زبان Strongly Typed و Statically Typed هست. یعنی چی؟

• Statically Typed: نوع هر متغیر باید در زمان کامپایل مشخص باشه.

• Strongly Typed: وقتی نوع متغیر مشخص شد، نمیتونی توش یه مقدار از نوع دیگه بریزی (مگر با تبدیل صریح).

انواع داده در جاوا به دو دسته بزرگ تقسیم میشن:

الف) انواع اولیه (Primitive Data Types)

این‌ها پایه‌ای‌ترین نوع‌ها هستن. توی Stack ذخیره میشن و مقدارشون مستقیماً توی متغیره. ۸ نوع هستن:

نوع	سایز	بازه	پیش فرض	مثال	نکته
byte	۱ بایت (۸ بیت)	128- تا 127	0	byte b = 100;	کم مصرف ترین نوع عددی. برای کار با فایل ها و Stream ها.
short	۲ بایت (۱۶ بیت)	32,768- تا 32,767	0	short s = 1000;	خیلی کم استفاده میشه.
int	۴ بایت (۳۲ بیت)	-2,147,483,648 تا 2,147,483,647 (حدود ۲ میلیارد)	0	int i = 50000;	پیش فرض و اصلی ترین نوع برای اعداد صحیح.
long	۸ بایت (۶۴ بیت)	خیلی بزرگ (۹ کوینتیلیون)	0L	long l = 100000L;	باید آخر عدد حرف L بذاری (کوچیک هم فرق نداره ولی بزرگش بهتره چون با ۱ اشتباه گرفته نشه).
float	۴ بایت (۳۲ بیت)	اعشاری با دقت حدود ۷ رقم	0.0f	float f = 5.6f;	باید آخر عدد حرف f بذاری. دقتش کمه، برای کارهای علمی دقیق خوب نیست.
double	۸ بایت (۶۴ بیت)	اعشاری با دقت حدود ۱۵ رقم	0.0d	double d = 5.6;	پیش فرض و اصلی ترین نوع برای اعداد اعشاری.
char	۲ بایت (۱۶ بیت)	یک کاراکتر یونیکد (Unicode)	'\u0000'	char c = 'A'; یا char c = 65;	تک کوت (') نه دابل کوت ("). میتونه عدد هم قبول کنه (کد ASCII).
boolean	۱~ بیت (وابسته به JVM)	true یا false	false	boolean flag = true;	فقط همین دو مقدار. برخلاف C، 0 و 1 نیست.

! سوال مصاحبه‌ای ۱: چرا $d = 0.1 + 0.2$ double برابر 0.3 نیست؟

جواب: به خاطر نحوه ذخیره‌سازی اعداد اعشاری بر اساس استاندارد IEEE 754. بعضی اعداد اعشاری دقیقاً در مبنای ۲ (باینری) قابل نمایش نیستن (مثل $1/3$ در مبنای ۱۰). برای کارهای مالی (مثل پول) هرگز از float و double استفاده نکن. همیشه از کلاس BigDecimal استفاده کن.

```
1 import java.math.BigDecimal;
2
3 // غلط برای پول
4 double bad = 0.1 + 0.2; // 0.30000000000000004
5
6 // درست برای پول
7 BigDecimal price = new BigDecimal("10.99");
8 BigDecimal tax = new BigDecimal("0.20");
9 BigDecimal total = price.add(price.multiply(tax)); // دقیق
```

! سوال مصاحبه‌ای ۲: Boxing و Unboxing چیه؟

جواب:

• Autoboxing: تبدیل خودکار یک نوع Primitive به کلاس Wrapper معادلش. مثلاً int به Integer.

• Unboxing: تبدیل خودکار کلاس Wrapper به Primitive معادلش.

```
1 Integer wrapper = 5; // Autoboxing: int -> Integer
```

```
2 int primitive = wrapper; // Unboxing: Integer -> int
```

نکته مهم: این کار هزینه داره و توی حلقه‌های بزرگ میتونه Performance رو بیاره پایین.
همچنین Integer میتونه null باشه ولی int نه، پس ممکنه موقع Unboxing به
NullPointerException بخوری.

ب) انواع مرجع (Reference Data Types)

هر چیزی که Primitive نیست، Reference Type هست: کلاس‌ها، اینترفیس‌ها، آرایه‌ها، Enum‌ها، String و ...

• نحوه ذخیره‌سازی: خود متغیر (Reference) توی Stack ذخیره میشه و به آدرس شیء توی Heap اشاره میکنه.

خود شیء توی Heap هست.

• مقدار پیش‌فرض: همیشه null.

```
1 String str; // است null هست و در Stack مقدار str
```

```
2 String str2 = new String("Hello"); // اشاره میکنه که داخلش Heap تو String هست، به یه شیء Stack تو str2  
هست "Hello".
```

! سوال مصاحبه‌ای ۳: تفاوت String با Primitive‌ها چیه؟ (چرا همه جا هست)

جواب: String یه کلاس Immutable (تغییرناپذیر) و یه Reference Type هست.

• Immutable: یعنی وقتی یه شیء String ساخته شد، محتوای دیگه عوض نمیشه. هر متدی که روش صدا بزنی

(مثل toUpperCase()) یه String جدید برمیگردونه.

•String Pool: یه منطقه خاص توی Heap هست. وقتی یه String رو بدون new بسازی ("String s = "hello")، جاوا اول استخر رو میگرد. اگه "hello" اونجا بود، همون reference رو برمیگردونه. این کار برای صرفه‌جویی در حافظه است.

```
1 String s1 = "hello";
2 String s2 = "hello";
3 System.out.println(s1 == s2); // true! اشاره میکنن String Pool چون هر دو به یک شیء توی
4
5 String s3 = new String("hello");
1 System.out.println(s1 == s3); // false! چون s3 با new از Pool مجبور شده یه شیء جدید بیرون
2 // استفاده کن equals همیشه برای مقایسه محتوا از
3 System.out.println(s1.equals(s3)); // true
```

! سوال مصاحبه‌ای ۴: تفاوت String، StringBuffer و StringBuilder چیست؟ (سوال طلایی)

جواب: هر سه برای کار با رشته‌ها هستن، اما تفاوت‌های کلیدی دارن:

ویژگی	String	StringBuilder	StringBuffer
تغییرپذیری	Immutable (غیرقابل تغییر)	Mutable (قابل تغییر)	Mutable (قابل تغییر)
امنیت ریس	✓ Thread-Safe	✗ ناامن	✓ Thread-Safe
سرعت	کندترین	سریع‌ترین	متوسط
استفاده	وقتی رشته تغییر نمی‌کنه	تکریمه‌ای	چندریمه‌ای

```
1 // String - غیرقابل تغییر
2 String s = "Hello";
```

```

3 s = s + " World"; // جدید ساخته میشه String به
4
5 // StringBuilder - سریع ولی ناامن
6 StringBuilder sb = new StringBuilder();
7 sb.append("Hello");
8 sb.append(" World"); // همان شیء تغییر میکنه
9
10 // StringBuffer - امن برای چند ریسه
11 StringBuffer sbf = new StringBuffer();
12 sbf.append("Hello");
13 sbf.append(" World");

```

1.2 عملگرها (Operators)

! سوال مصاحبه‌ای ۴: فرق << و <<< چیه؟ (شیفت بیتی)

جواب: اینا عملگرهای شیفت بیتی هستن و کمتر پیش میاد، ولی خوبه بدونی:

- << (شیفت به راست علامت‌دار): بیت علامت (چپ‌ترین بیت) رو حفظ میکنه. برای اعداد منفی، ۱ به چپ اضافه میشه.

- <<< (شیفت به راست بدون علامت): صفر به چپ اضافه میکنه. نتیجه همیشه مثبته.

```

16 ``java
17 int negative = -8; // 1000...1111 باینری:
18 int signedShift = negative >> 2; // 1111...1110 = -2
19 int unsignedShift = negative >>> 2; // 0011...1110 = 1073741822

```

! سوال مصاحبه‌ای ۵: & و && (و همینطور | و ||) چه فرقی دارن؟

جواب: این خیلی مهمه:

• & و | : همیشه هر دو طرف عبارت رو ارزیابی میکنن (هم برای boolean، هم bitwise).

• && و || : ارزیابی اتصال کوتاه (Short-circuit) دارن. یعنی اگر با ارزیابی سمت چپ، نتیجه کل عبارت مشخص بشه، سمت راست اصلاً اجرا نمیشه.

```
1 // a. یک متد سنگین یا دارای عوارض جانبی است getValue() فرض کنید
2 if (false && getValue()) { // b. هرگز فراخوانی نمی‌شود getValue() تابع
3 }
4 if (false & getValue()) { // c. getValue() حتی اگر نتیجه
5 }
```

1.3 ساختارهای کنترلی (Control Flow)

! سوال مصاحبه‌ای ۶: switch-case قدیم و جدید (Enhanced switch) چه فرقی دارن؟

(+Java 14)

جواب:

• قدیم (Statement): مثل ++C/C هست. نیاز به break داره، وگرنه میافته توی case بعدی (fall-through).

• جدید (Expression): میتونه یه مقدار برگردونه. از علامت -> استفاده میکنه. نیازی به break نداره. خیلی تمیزتر و کم‌خطاتر هست.

```

1 // a. (بهتر)
2 String dayType = switch (day) {
1   case MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY -> "Weekday";
2   case SATURDAY, SUNDAY -> "Weekend";
3   // b. لازم نیست default استفاده کنی و همه حالات رو پوشش بدی، دیگه enum اگه
4 };

```

! سوال مصاحبه‌ای ۷: فرق for، for-each، و while در جاوا؟ بهترین زمان استفاده از هرکدام؟

جواب:

• for سنتی: وقتی به ایندکس (شماره خونه) نیاز داری. (مثلاً میخوای عنصر $i+1$ ام رو با i مقایسه کنی).

• for-each (enhanced for): وقتی فقط میخوای همه عناصر یه مجموعه رو بخونی. خوندنی‌ترین و امن‌ترین حالت پیمایشه. نمیتونی توش عناصر رو حذف یا تغییر بدی (فقط خوندنیه).

• while: وقتی تعداد تکرار مشخص نیست و به یه شرط وابسته است (مثلاً تا وقتی که کاربر دستور exit نزده).

• do-while: مثل while، اما تضمین میکنه که بدنه حلقه حداقل یک بار اجرا بشه.

1.4 آرایه‌ها (Arrays)

سوال: آرایه توی جاوا چطوریه؟

• یه ساختمان داده با طول ثابت (Fixed Size) که تعدادی خونه پشت سر هم توی حافظه Heap داره.

•نوع همه خونه‌هاش باید یکی باشه.

•سایزش بعد از ساخته شدن دیگه عوض نمیشه. اگه سایز داینامیک میخوای، باید بری سراغ ArrayList.

```

1 // سه روش برای ساختن آرایه a.
1 int[] arr1 = new int[5]; // b. همه خونه‌ها مقدار پیش‌فرض 0 دارن
2 int[] arr2 = {1, 2, 3, 4, 5}; // c. سایز خودکار 5 میشه
3 int[] arr3 = new int[] {1, 2, 3}; // d. یه جور دیگه
4
5 // آرایه دو بعدی (آرایه‌ای از آرایه‌ها) a.
6 int[][] matrix = {
7     {1, 2, 3},
8     {4, 5, 6}
9 };
10 System.out.println(matrix[0][1]); // خروجی: 2 b.

```

! سوال مصاحبه‌ای ۸: ArrayIndexOutOfBoundsException کی رخ میده؟

جواب: وقتی سعی کنی به یه ایندکس خارج از بازه آرایه دسترسی پیدا کنی (مثلاً آرایه ۵ تایی، ایندکس ۵). در جاوا ایندکس‌ها از صفر شروع میشن. این یه Unchecked Exception هست.

1.5 آرگومان‌های خط فرمان (args)

سوال: args []String توی متد main چیه؟ جواب: هر چیزی که موقع اجرای برنامه جاوا از خط فرمان بعد از اسم برنامه بنویسی، توی این آرایه قرار میگیره.

1 java MyProgram arg1 arg2

2

3 هست String همیشه از نوع "arg2" میشه [1]args و "arg1" میشه [0]args، توی برنامه